DFSha: diseño e implementación de un sistema de archivos distribuido con alta disponibilidad, rendimiento y seguridad

Alberto Daniel Cervantes Forero y Andres Felipe Garnica Roa
Escuela de Ciencias Aplicadas e Ingeniería, Universidad EAFIT
Sistemas Distribuidos
Álvaro Ospina Sanjuán

Resumen

Este informe presenta el diseño, la implementación y la evaluación de DFSha, un sistema de archivos distribuido por bloques desarrollado como primer proyecto del curso de Sistemas Distribuidos. Siguiendo el modelo de HDFS, el sistema tiene un ControlNode, que administra los metadatos, la autenticación y los permisos, y tres DataNodes que almacenan los bloques. Cada archivo se divide en bloques de 64 MB que se cifran con Fernet, se reparten con una estrategia Round Robin y se guardan dos veces en nodos distintos. Todas las comunicaciones usan REST sobre HTTPS, y el acceso se controla con tokens JWT, permisos de tipo Unix, listas de control de acceso y grupos. La consistencia se garantiza escribiendo versiones completas que solo se activan cuando terminan, junto con un bloqueo exclusivo de escritura por archivo. El sistema se desplegó en cuatro máquinas virtuales de AWS Academy usando Docker. Las 161 pruebas automáticas y la prueba de despliegue se aprobaron en su totalidad; al detener un DataNode, el sistema detectó la falla en 30 s, restauró las copias perdidas y el archivo se descargó íntegro. La principal limitación es que el ControlNode no está replicado, por lo que la alta disponibilidad alcanzada es la de los datos.

Palabras clave: sistemas de archivos distribuidos, HDFS, replicación, alta disponibilidad, tolerancia a fallos, cifrado, computación en la nube

Contenido

Objetivo y marco teórico 6
Objetivo 6
Qué es un DFS 6
GFS y HDFS 6
Dónde queda DFSha 6
Descripción del servicio 6
Problema 6
Arquitectura asignada 7
Arquitectura del sistema 8
Tipo de arquitectura 8
Por qué un maestro central 9
Protocolos de comunicación y APIs 9
Por qué el cliente no habla directo con los DataNodes 10
Particionamiento, distribución y replicación 11
Particionamiento 11
Distribución con Round Robin 11
Subida de un archivo (PUT) 12
Descarga (GET) 12
Replicación 12
Consistencia y concurrencia 12
Versiones 13
WRITE LOCK 13
Lecturas concurrentes 13
Estados del archivo 13

Decisión documentada 13
Seguridad 13
Control de acceso 14
Alta disponibilidad y tolerancia a fallos 15
Monitor de DataNodes 15
Re-replicación 16
Limpieza periódica 16
Lectura con un nodo caído 16
Entorno de ejecución 16
Tecnologías 16
Desarrollo local 17
Despliegue en AWS Academy 17
Security groups 17
Contenedores en ejecución 19
API publicada 19
Pruebas y análisis de resultados 20
Pruebas automáticas 21
Errores que encontraron las pruebas 22
Prueba del despliegue en AWS 25
Caída de un DataNode en AWS 26
Procedimiento 26
Resultado 27
Detección 28
Integridad 31
Recuperación 31

Análisis 32

Limitaciones y trabajo futuro 32

Conclusiones 33

Referencias 34

Objetivo y marco teórico

Objetivo

Diseñar e implementar un sistema de archivos distribuido (DFS) que guarde archivos grandes repartidos en varios nodos, con alta disponibilidad, buen rendimiento, transparencia para el cliente y seguridad en las comunicaciones y en el almacenamiento (Universidad EAFIT, 2026).

Qué es un DFS

Es un sistema que deja guardar y leer archivos que en realidad viven en varias máquinas, pero que el usuario ve como un solo árbol de carpetas. NFS, AFS y SMB son los ejemplos clásicos (Universidad EAFIT, 2026). Hay dos familias:

- *Por bloques:* la unidad de lectura y escritura es el bloque, y los bloques de un mismo archivo quedan en nodos distintos.
- *Por objetos* (como AWS S3): la unidad es el archivo completo, que se escribe una vez y se lee muchas (WORM). No hay actualización parcial: para cambiar algo se reemplaza el archivo entero.

GFS y HDFS

Google File System y su versión abierta, Hadoop Distributed File System, usan un nodo maestro con los metadatos (NameNode) y muchos nodos de datos (DataNodes). Los archivos se parten en bloques grandes (64 o 128 MB), cada bloque se replica en varios DataNodes y el maestro detecta por *heartbeats* cuándo un nodo cae para volver a replicar sus bloques (Ghemawat et al., 2003; Shvachko et al., 2010).

Dónde queda DFSha

Tomamos el modelo de HDFS: un ControlNode con los metadatos y tres DataNodes con bloques de 64 MB y factor de replicación 2. Por dentro es un DFS por bloques (los archivos se parten y se reparten), pero hacia el cliente se comporta parecido a un almacenamiento de objetos: se accede con un CLI y una API REST, no se monta en el sistema operativo, y el `write` reemplaza el archivo completo por una versión nueva.

Descripción del servicio

Problema

Un solo servidor no alcanza para guardar archivos muy grandes ni sobrevive si se daña. DFSha reparte cada archivo en bloques entre varios nodos y guarda cada bloque dos veces, así

que se puede perder un nodo sin perder datos. Todo con usuarios, permisos y cifrado, porque los archivos son de cada usuario.

Arquitectura asignada

Opción 1, cliente/servidor. El servicio es autónomo (lo diseñamos y operamos nosotros), corre en una red privada (la VPC de AWS) y expone una API abierta al cliente (Tablas 1 y 2).

Tabla 1

Requisitos funcionales de DFSha

Código	Requisito	Cómo se cumple (comando del CLI)
RF1	Gestión del sistema de archivos	`fs ls`, `fs cd`, `fs mkdir`, `fs rmdir`, `files rm`
RF2	Transferencia de archivos (send / receive)	`files put` (con `--overwrite` para reemplazar) y `files get`
RF3	Acceso a archivos	`files open` (READ o WRITE, este último toma el lock), `files read`, `files write`, `files close`
RF4	Usuarios y autenticación	`auth register`, `auth login` con usuario y contraseña; cada usuario recibe su `/home/<usuario>`
RF5	Control de acceso por usuario y grupo	`access chmod`, `access chgrp`, `access acl-set`, `access acl-rm`, `access group-create`, `access group-add`

Tabla 2

Requisitos no funcionales de DFSha

Requisito	Cómo lo resuelve DFSha
Particionamiento	Bloques de 64 MB repartidos con Round Robin entre los DataNodes en línea
Alta disponibilidad	Factor de replicación 2 (PRIMARY + BACKUP en nodos distintos), monitor de salud y re-replicación automática
Consistencia	Escrituras como versiones nuevas que se activan solo cuando están completas; WRITE LOCK por archivo
Rendimiento y concurrencia	Descarga de bloques en paralelo (hasta 8), lecturas simultáneas, archivos leídos por partes sin cargarlos enteros en memoria
Escalabilidad	Agregar un DataNode es agregarlo a la variable `DATANODES`; el reparto lo incluye solo
Seguridad	Cifrado en reposo (Fernet) y en tránsito (HTTPS en todos los saltos), JWT, permisos Unix + ACL, clave interna entre nodos
Transparencia	El cliente usa rutas tipo Linux (`/home/beto/informe.pdf`) y nunca sabe en qué nodo está cada bloque

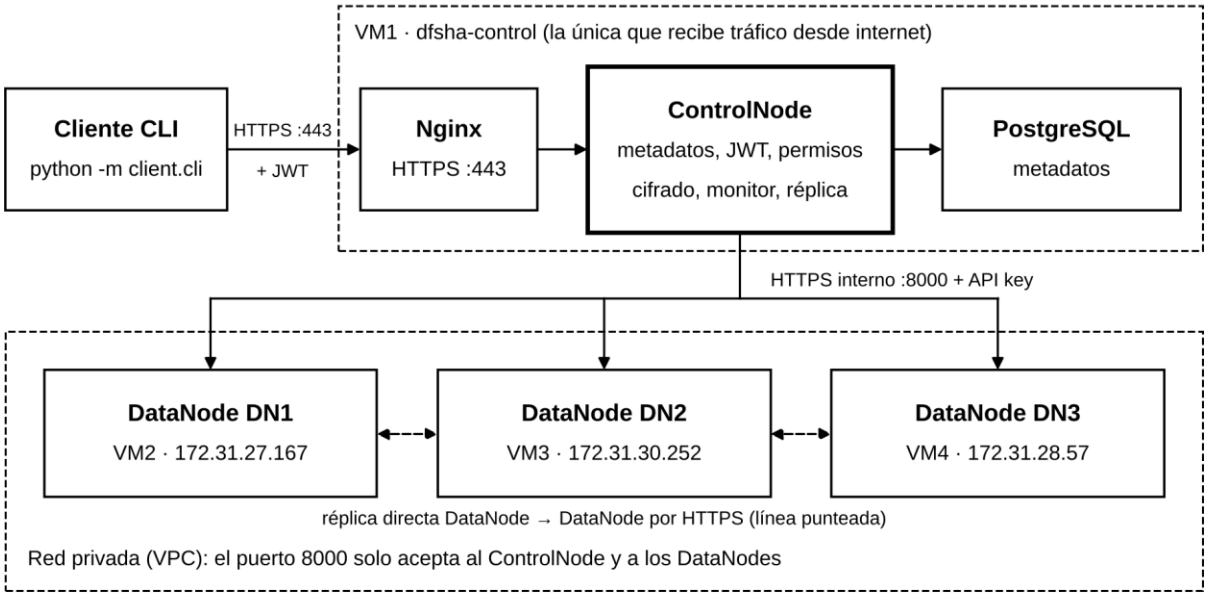
Arquitectura del sistema

Tipo de arquitectura

La arquitectura es de tipo maestro–trabajadores (*master–workers*) cliente/servidor, como HDFS. El ControlNode es el maestro: guarda los metadatos y decide. Los DataNodes son los trabajadores: solo guardan y entregan bloques (Figura 1).

Figura 1

Arquitectura de DFSha desplegada en AWS



Nota. El cliente solo se comunica con Nginx en la VM1; los DataNodes están en la red privada de AWS y no son accesibles desde internet.

El cliente solo conoce la IP pública de la VM1. Todo lo demás viaja por la red privada de AWS, cifrado (Tabla 3).

Tabla 3

Componentes del sistema y su responsabilidad

Componente	Responsabilidad
Cliente CLI (`client/`)	Comandos del usuario (Typer). Guarda la sesión (URL y JWT) en `~/.dfsha/session.json` y habla solo con Nginx
Nginx (`nginx/`)	Termina HTTPS hacia el cliente, redirige el puerto 80 a 443 y reenvía al ControlNode sin límite de tamaño de subida

Componente	Responsabilidad
ControlNode (`control_node/`)	Autenticación, árbol de directorios, permisos, locks, partición y distribución de bloques, cifrado, monitor de DataNodes, re-replicación y limpieza. Es el único que habla con el cliente
PostgreSQL	Metadatos: usuarios, grupos, directorios, archivos y sus versiones, bloques, ubicación de cada réplica, ACL, handles y locks
DataNode (`data_node/`)	Guarda, entrega, borra y copia bloques por su id, y verifica su SHA-256. Es simple a propósito: no sabe de archivos ni de usuarios
Compartido (`shared/`)	Constantes, enums y el certificado interno, comunes al ControlNode y a los DataNodes

Por qué un maestro central

Tener los metadatos en un solo lugar hace fácil mantenerlos consistentes (una transacción de PostgreSQL decide qué versión de un archivo es la válida) y deja a los DataNodes como piezas intercambiables: agregar uno es solo declararlo en `DATANODES`.

Protocolos de comunicación y APIs

Todo el sistema habla REST sobre HTTPS (JSON y `multipart/form-data` para los bytes). Elegimos REST porque cada nodo expone su API con FastAPI, se prueba con `curl` y queda documentada sola en `/docs` (Swagger) (Tabla 4).

Tabla 4

Protocolos de comunicación entre los componentes

Comunicación	Protocolo	Autenticación	Para qué
Cliente ↔ ControlNode	HTTPS (puerto 443, Nginx)	JWT de usuario (`Authorization: Bearer`)	Todas las operaciones del usuario
ControlNode ↔ DataNode	HTTPS interno (puerto 8000)	Cabecera `X-DFSHA-API-Key` + certificado interno	Guardar, leer, borrar y listar bloques; health checks; órdenes de réplica
DataNode ↔ DataNode	HTTPS interno (puerto 8000)	Igual que la anterior	Copiar un bloque directo de un nodo a otro durante la re-replicación
Cliente ↔ DataNode	No existe, por diseño	—	El cliente nunca ve un DataNode (véase el apartado siguiente)
ControlNode ↔ ControlNode	No aplica	—	Hay un solo ControlNode (véase la sección Limitaciones y trabajo futuro)

Por qué el cliente no habla directo con los DataNodes

En HDFS el cliente sí lee directamente de los DataNodes (Shvachko et al., 2010). Aquí decidimos que todo pase por el ControlNode porque es el que cifra y descifra los bloques: así los DataNodes solo guardan bytes cifrados, nunca quedan expuestos a internet (el security group los cierra) y la clave de cifrado vive en un solo lugar. El costo es que el ControlNode carga con el tráfico de datos; lo compensamos leyendo los bloques en paralelo.

Todas las rutas de la API del ControlNode exigen JWT, salvo el registro, el inicio de sesión, `/health` y `/datanodes` (Tabla 5).

Tabla 5

Rutas de la API del ControlNode

Método y ruta	Operación
`POST /auth/register`, `POST /auth/login`, `GET /auth/me`	Crear cuenta, iniciar sesión (devuelve el JWT)
`POST /directories`, `DELETE /directories`	mkdir, rmdir
`GET /filesystem/list`, `GET /filesystem/resolve`	ls, cd
`POST /files` (multipart: `directory_path`, `file`, `overwrite`)	put. 201 creado, 409 si ya existe sin overwrite, 423 si está bloqueado, 422 si no hay factor 2 posible
`GET /files/download?path=`	get, en streaming
`DELETE /files`	rm
`POST /files/open`, `/files/read`, `/files/write`, `/files/close`	open, read, write, close (handle por sesión)
`GET /access`, `PATCH /access/mode`, `PATCH /access/group`	Ver permisos, chmod, chgrp
`PUT /access/acl`, `DELETE /access/acl`	Dar o quitar ACL a un usuario o grupo
`POST /access/groups`, `POST /access/groups/{nombre}/members`	Crear grupos y agregar miembros (por nombre)
`POST /groups`, `GET /groups`, `POST /groups/{id}/members`	Lo mismo por id de grupo y de usuario; solo un miembro puede agregar a otros
`GET /datanodes`, `GET /health`	Estado de los DataNodes y del servicio

La API del DataNode es interna y todas sus rutas, salvo `/health`, exigen la cabecera `X-DFSHA-API-Key` (Tabla 6).

Tabla 6

Rutas de la API interna del DataNode

Método y ruta	Operación
`POST /blocks`	Guardar un bloque (`block_id`, `checksum`, contenido); verifica el SHA-256 antes de aceptarlo
`GET /blocks/{id}`	Entregar un bloque en streaming
`DELETE /blocks/{id}`	Borrar un bloque (idempotente: 204 aunque ya no exista)
`GET /blocks`	Listar los bloques guardados (para encontrar copias huérfanas)
`POST /blocks/{id}/replicate`	Copiar el bloque a otro DataNode
`GET /health`	Nombre del nodo, estado y espacio libre

La documentación interactiva completa de la API quedó publicada en `https://<IP-VM1>/docs`; las capturas se presentan en la sección Entorno de ejecución.

Particionamiento, distribución y replicación

Particionamiento

Cada archivo se corta en bloques de 64 MB (`BLOCK_SIZE_MB`); el último bloque puede ser más pequeño. El ControlNode lee el archivo subido por partes de 64 MB, así que nunca lo carga entero en memoria.

Distribución con Round Robin

Para el bloque número *i*, la lista de DataNodes elegibles se rota empezando en la posición *i mod N*: el primero de la lista recibe la réplica PRIMARY y el siguiente la BACKUP. Un nodo es elegible si está `ONLINE` y tiene espacio para el bloque. Como la rotación depende solo del índice, el reparto es parejo y se puede predecir. Con 3 nodos y un archivo de 300 MB (5 bloques) queda así (Tabla 7):

Tabla 7

Distribución Round Robin de un archivo de cinco bloques en tres DataNodes

Bloque	PRIMARY	BACKUP
B0	DN1	DN2
B1	DN2	DN3
B2	DN3	DN1
B3	DN1	DN2
B4	DN2	DN3

Eso da 3 copias en DN1, 4 en DN2 y 3 en DN3, exactamente lo que medimos en AWS (véase la sección Pruebas y análisis de resultados). La estrategia está detrás de una interfaz (`DistributionStrategy`), así que se puede cambiar por una basada en espacio libre o carga sin tocar el resto.

Subida de un archivo (PUT)

El proceso sigue estos pasos:

1. Valida que el usuario pueda escribir en el directorio y que haya al menos 2 DataNodes elegibles. Si no los hay, rechaza la subida (422): nunca se guarda un bloque con una sola copia.
2. Crea el archivo en estado `UPLOADING`.
3. Por cada bloque: lo cifra, calcula el SHA-256 del bloque cifrado, elige PRIMARY y BACKUP, y lo envía a los dos nodos.
4. Si una réplica falla, conserva la que sí quedó y prueba en *otro DataNode elegible*, hasta `MAX_RETRIES` (3) intentos. Si no lo logra, la subida falla y se borran los bloques que alcanzaron a guardarse.
5. Cuando todos los bloques tienen sus 2 copias, el archivo pasa a `AVAILABLE` y aparece en `ls`.

Descarga (GET)

El ControlNode baja los bloques en paralelo (hasta `MAX_PARALLEL_DOWNLOADS` = 8 a la vez), primero de la réplica PRIMARY y, si falla o su checksum no coincide, de la BACKUP. Verifica el SHA-256 de cada bloque, lo descifra y arma el archivo en una carpeta temporal; antes de empezar reserva el espacio necesario más un 10 % de margen, y si no alcanza responde 507. El archivo se entrega en *streaming*.

Replicación

El factor es 2 y es obligatorio: PRIMARY y BACKUP siempre están en nodos distintos. Si un nodo cae, sus bloques se vuelven a copiar a otro nodo para recuperar las 2 copias (véase la sección Alta disponibilidad y tolerancia a fallos).

Consistencia y concurrencia

La regla central es que un archivo nunca queda a medio escribir a la vista del usuario: o se ve la versión vieja completa o la nueva completa.

Versiones

Cada bloque guarda el número de versión del archivo al que pertenece. Un `write` o un `put --overwrite` no modifica bloques sueltos: guarda el contenido nuevo como una versión nueva (`next_version`). Solo cuando todos sus bloques tienen sus 2 copias, el archivo cambia su `current_version` en una sola transacción de PostgreSQL, y después se borran los bloques de la versión vieja. Si algo falla a mitad de camino, la versión vieja sigue activa y la nueva se descarta.

WRITE LOCK

El lock es exclusivo y de archivo completo:

- Lo toman `open` en modo WRITE y `put --overwrite`. La tabla `file_locks` tiene una restricción única por archivo y la fila se bloquea con `SELECT ... FOR UPDATE`, así que dos sesiones no pueden ganarlo a la vez.
- Mientras está tomado, cualquier otro `open`, `read`, GET, `rm` u overwrite responde *423 Locked*.
- Vence a los `LOCK_TIMEOUT_SECONDS` (30 s) y cada escritura lo renueva. Si el cliente desaparece sin cerrar, el lock se libera solo al vencer y el handle viejo responde *410*.

Lecturas concurrentes

Varias sesiones pueden leer el mismo archivo a la vez (modo READ), porque leer no cambia nada. Un handle de lectura vence tras `HANDLE_TIMEOUT_SECONDS` (300 s) sin uso.

Estados del archivo

`UPLOADING` → `AVAILABLE`, y `DELETING` o `FAILED` cuando se borra o una subida falla. Solo los `AVAILABLE` se ven en `ls` y se pueden abrir. Al borrar un archivo se le cambia el nombre interno (`.~dfsha-borrando-<id>`) para que el usuario pueda volver a subir uno con el mismo nombre aunque la limpieza siga pendiente. Si el ControlNode se reinicia en medio de una subida, al arrancar marca esa subida como `FAILED` y la limpia.

Decisión documentada

El `write` recibe un *offset*, pero reemplaza el archivo completo. Es más costoso para archivos enormes, pero garantiza consistencia sin locks por bloque ni archivos con huecos.

Seguridad

Los datos van cifrados en los tres lugares donde podrían verse: en el disco, entre el cliente y el servidor, y entre los nodos (Tabla 8).

Tabla 8

Mecanismos de seguridad por capa

Capa	Mecanismo	Detalle
Datos en reposo	Fernet (AES-128-CBC + HMAC-SHA256)	El ControlNode cifra cada bloque antes de enviarlo; los DataNodes solo guardan bytes cifrados. La clave (`ENCRYPTION_KEY`) vive solo en el ControlNode
Integridad	SHA-256 por bloque	Se calcula sobre el bloque cifrado; el DataNode lo verifica al guardar y el ControlNode al leer. Un bloque dañado se descarta y se usa la otra réplica
Cliente ↔ servidor	HTTPS con Nginx	Solo el puerto 443 está abierto; el 80 redirige a HTTPS (301). Certificado autofirmado generado en la VM
ControlNode ↔ DataNode y DataNode ↔ DataNode	HTTPS interno	Certificado Ed25519 derivado de `INTERNAL_API_KEY` con HKDF-SHA256. Todos los nodos obtienen el mismo certificado sin copiar archivos, y el ControlNode solo confía en ese certificado. Un DataNode no acepta HTTP sin cifrar
Autenticación entre nodos	Clave interna	Cabecera `X-DFSHA-API-Key`, comparada en tiempo constante; sin ella el DataNode responde 401
Usuarios	Usuario y contraseña + JWT	Contraseñas con Argon2; el login entrega un JWT que vence en 1 hora
Autorización	Permisos Unix + ACL + grupos	Ver abajo
Red	Security groups de AWS	Los DataNodes no son públicos: el puerto 8000 solo acepta tráfico del ControlNode y de los otros DataNodes

Control de acceso

Cada archivo y directorio tiene dueño, grupo y bits rwx para dueño, grupo y otros (por defecto 640 para archivos). Encima se pueden poner ACL por usuario o por grupo. Para decidir si alguien puede leer, escribir o atravesar, el ControlNode revisa en este orden:

1. Bits del dueño, si es el dueño (como en las ACL POSIX: una ACL de un grupo al que el dueño pertenece no puede dejarlo sin acceso a lo suyo).
2. ACL del usuario sobre ese recurso.
3. ACL de sus grupos (basta con que uno lo permita).
4. Bits del grupo, si pertenece al grupo del recurso.

5. Bits de otros.

En un directorio, *r* es listar, *w* es crear o borrar dentro y *x* es atravesarlo. Solo el dueño cambia permisos (`chmod`, `chgrp`, ACL), y solo puede asignar un grupo al que pertenece. Quien crea un grupo queda como miembro, y solo los miembros pueden agregar a otros; así nadie puede meterse solo a un grupo ajeno para quedarse con sus permisos.

Alta disponibilidad y tolerancia a fallos

Se puede perder cualquier DataNode sin perder datos ni dejar de atender: cada bloque tiene otra copia, y el sistema la vuelve a duplicar en otro nodo en cuanto detecta la caída.

Monitor de DataNodes

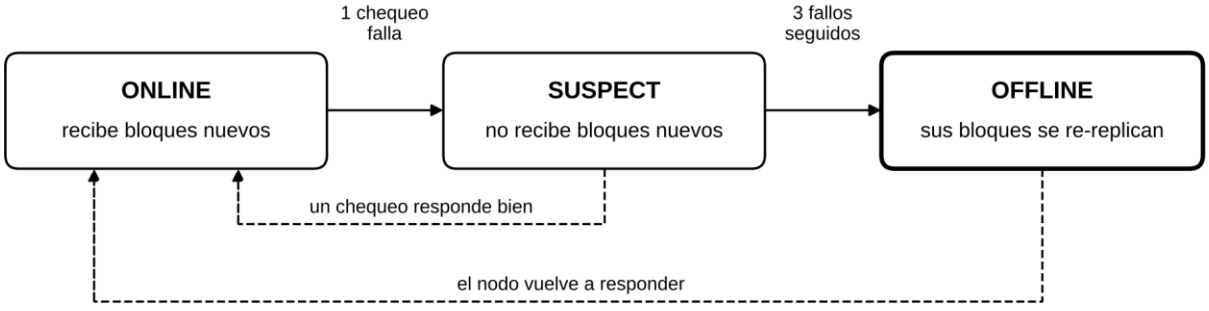
El ControlNode consulta `GET /health` de todos los DataNodes al mismo tiempo cada `HEALTH_CHECK_INTERVAL_SECONDS` (10 s). Cada nodo pasa por tres estados:

- *ONLINE:* respondió bien. Puede recibir bloques nuevos.
- *SUSPECT:* falló 1 o 2 chequeos seguidos. No recibe bloques nuevos, pero todavía no se considera caído (puede ser un problema de red pasajero).
- *OFFLINE:* falló `DATANODE_FAILURE_THRESHOLD` (3) chequeos seguidos. Ahí arranca la re-replicación.

Un chequeo correcto devuelve el nodo a ONLINE. El chequeo también valida que conteste el nodo esperado (`node_id`) y guarda su espacio libre, que se usa para decidir dónde poner bloques. Con estos valores, una caída se detecta en unos 30 segundos (Figura 2).

Figura 2

Estados de un DataNode en el monitor del ControlNode



Nota. El ControlNode consulta /health de cada DataNode cada 10 s; un nodo pasa a OFFLINE después de 3 chequeos fallidos seguidos y entonces se re-replican sus bloques.

Re-replicación

Cuando un nodo pasa a OFFLINE, el ControlNode busca los bloques que tenían una copia ahí. Para cada uno elige un nodo ONLINE que no tenga ese bloque y le ordena al nodo que sí tiene la copia buena que se la pase directamente (DataNode → DataNode, por HTTPS). El nodo de destino verifica el SHA-256 antes de guardarla. Esto corre en segundo plano, así que el ControlNode sigue atendiendo mientras tanto; en las pruebas automáticas, con 80 copias por recuperar, la respuesta más lenta del API fue de 0,07 s.

Limpieza periódica

Cada `CLEANUP_INTERVAL_SECONDS` (30 s), una tarea de fondo:

- libera locks y handles vencidos;
- termina los borrados que quedaron pendientes porque un nodo estaba caído;
- borra los bloques de subidas fallidas y de versiones viejas;
- vuelve a intentar la re-replicación de bloques que siguen con una sola copia;
- borra las *copias huérfanas*: bloques que un nodo todavía guarda pero que los metadatos ya no registran, por ejemplo las copias viejas de un nodo que volvió después de ser re-replicado. Solo borra las que tienen más de `ORPHAN_GRACE_SECONDS` (15 min), para no tocar una subida en curso.

Lectura con un nodo caído

El GET no espera a la re-replicación: si la PRIMARY no responde, usa la BACKUP. Por eso el archivo se puede descargar incluso en los segundos en que el nodo está caído y todavía no se ha recuperado nada.

Entorno de ejecución

Cada componente corre en un contenedor Docker. Usamos el mismo código en dos entornos: un clúster completo en un solo computador para desarrollar, y 4 máquinas virtuales en AWS Academy para el despliegue real.

Tecnologías

El código fuente está publicado en GitHub (Garnica Roa y Cervantes Forero, 2026). Se usaron Python 3.12, FastAPI y Uvicorn, SQLAlchemy y Alembic (migraciones), PostgreSQL 16, httpx, cryptography (Fernet, Ed25519, HKDF), Typer (CLI), Nginx 1.27, Docker y Docker Compose.

Desarrollo local

El `docker-compose.yml` de la raíz levanta PostgreSQL, el ControlNode, tres DataNodes y Nginx en la misma máquina, cada DataNode con su volumen. Se usa para programar y para las pruebas automáticas (véase la sección Pruebas y análisis de resultados).

Despliegue en AWS Academy

Las cuatro máquinas virtuales están en la región us-east-1, usan Ubuntu 24.04 y comparten la misma VPC y subred (Tabla 9).

Tabla 9

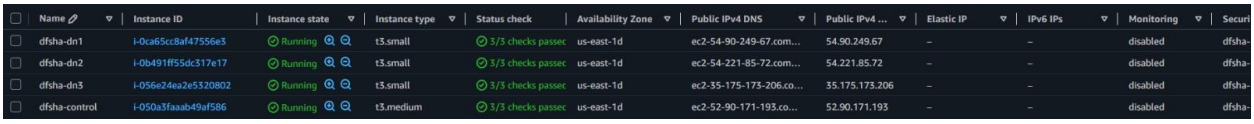
Máquinas virtuales del despliegue en AWS Academy

VM	Nombre	Tipo	Qué corre	IP privada
VM1	`dfsha-control`	t3.medium (2 vCPU, 4 GB)	ControlNode + PostgreSQL + Nginx	172.31.22.9
VM2	`dfsha-dn1`	t3.small (2 vCPU, 2 GB)	DataNode DN1	172.31.27.167
VM3	`dfsha-dn2`	t3.small	DataNode DN2	172.31.30.252
VM4	`dfsha-dn3`	t3.small	DataNode DN3	172.31.28.57

Los nodos se comunican por la red privada. Los bloques de cada DataNode se guardan en `/data/dfsha` de su VM, fuera del contenedor, así que sobreviven si el contenedor se reconstruye. Todos los contenedores tienen `restart: unless-stopped`: al prender las VMs, el sistema arranca solo. Los pasos completos están en `deploy/README.md` del repositorio (Figura 3).

Figura 3

Instancias EC2 del despliegue en ejecución



Nota. Las 4 VMs en estado Running con 3/3 chequeos aprobados.

Security groups

Hay dos: `dfsha-control` para la VM1 y `dfsha-datanode` para las otras tres (Tabla 10).

Tabla 10

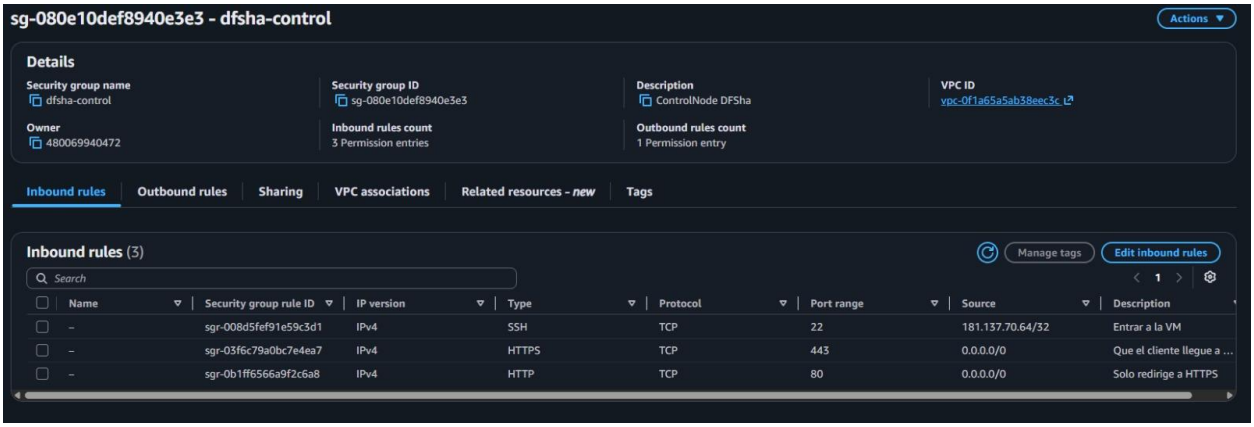
Reglas de entrada de los security groups

Grupo	Puerto	Origen	Para qué
`dfsha-control`	22	IP del equipo	Administrar por SSH
`dfsha-control`	443	0.0.0.0/0	Que el cliente llegue a Nginx por HTTPS
`dfsha-control`	80	0.0.0.0/0	Solo redirige a HTTPS
`dfsha-datanode`	22	IP del equipo	Administrar por SSH
`dfsha-datanode`	8000	grupo `dfsha-control`	Que el ControlNode hable con los DataNodes
`dfsha-datanode`	8000	grupo `dfsha-datanode`	Que los DataNodes se copien bloques entre ellos

El puerto 8000 nunca se abre a internet: solo lo alcanzan las VMs del propio sistema (Figuras 4 y 5).

Figura 4

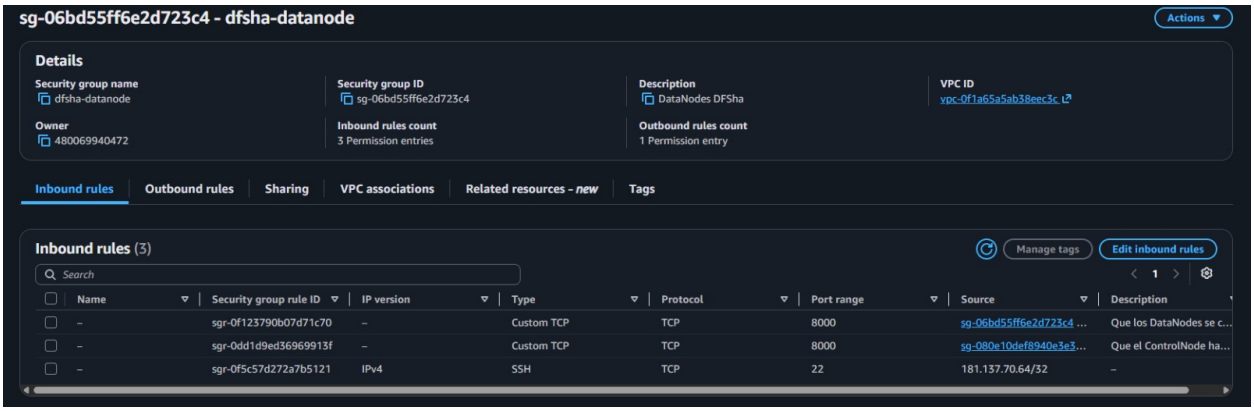
Reglas de entrada del security group dfsha-control



Nota. Reglas de entrada de `dfsha-control`.

Figura 5

Reglas de entrada del security group dfsha-datanode



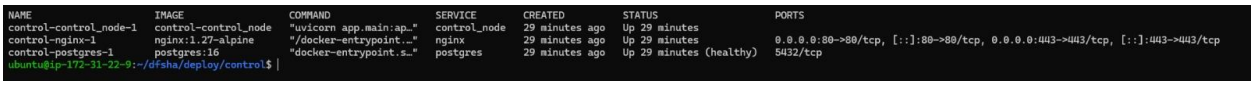
Nota. Reglas de entrada de `dfsha-datanode`: el puerto 8000 solo acepta los dos security groups del sistema.

Contenedores en ejecución

En la VM1 corren el ControlNode, Nginx y PostgreSQL; en cada VM de DataNode corre un solo contenedor (Figuras 6 y 7).

Figura 6

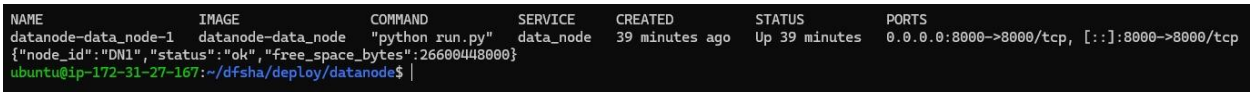
Contenedores en ejecución en la VM1



Nota. VM1: `control_node`, `nginx` (puertos 80 y 443) y `postgres` (healthy).

Figura 7

Contenedor del DataNode DN1 en la VM2



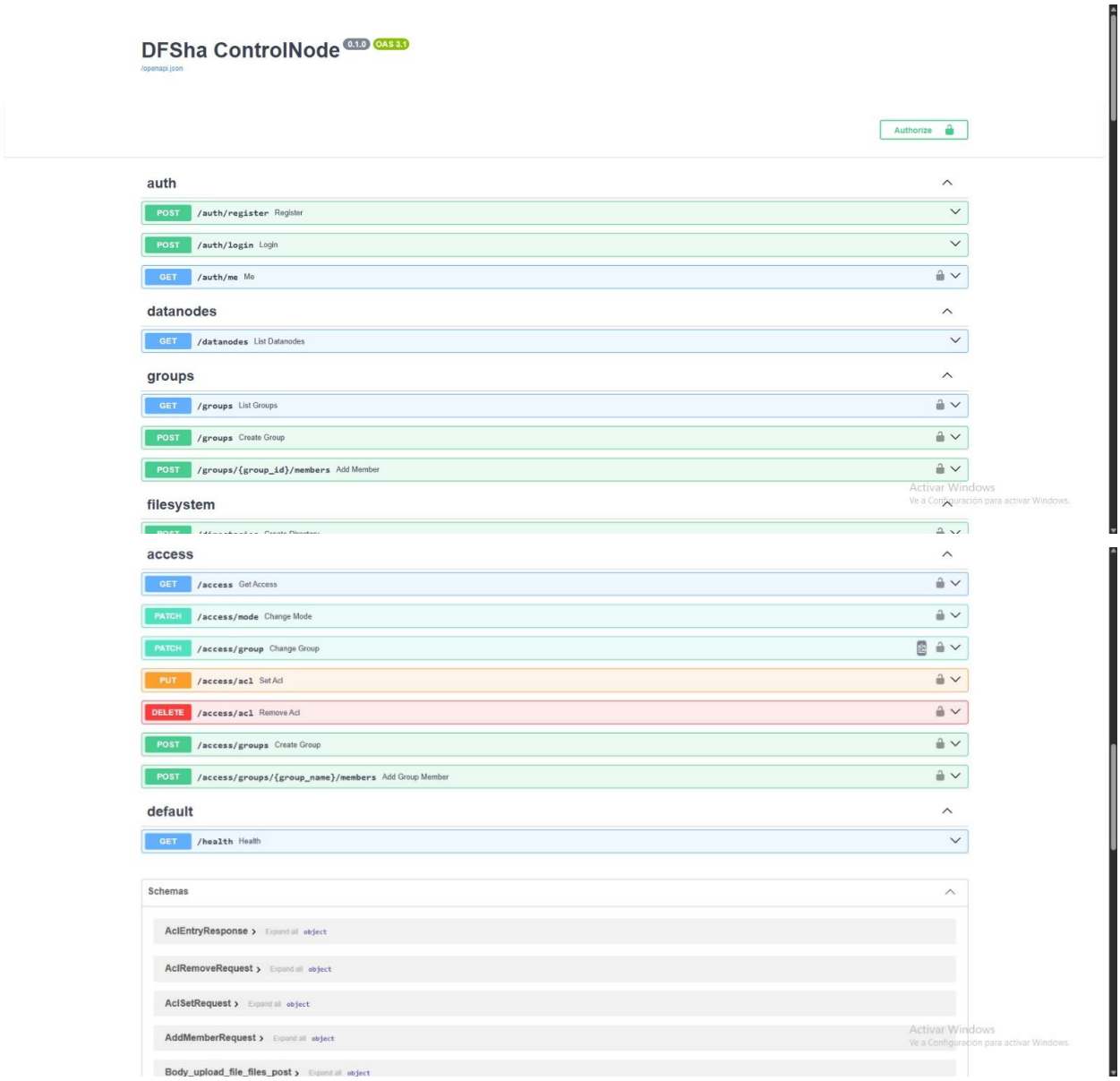
Nota. VM2: el DataNode corre con `python run.py` y responde `/health` por HTTPS.

API publicada

La documentación interactiva (Swagger) queda disponible en `https://<IP-pública-VM1>/docs` (Figura 8).

Figura 8

Documentación interactiva de la API del ControlNode



Nota. Endpoints del ControlNode en Swagger, servidos por HTTPS desde AWS.

Pruebas y análisis de resultados

Todas las pruebas pasaron. Probamos en dos niveles: suites automáticas en el entorno local, y en AWS una prueba de despliegue más una caída real de un DataNode.

Pruebas automáticas

Las pruebas están en la carpeta `tests/` del repositorio (Garnica Roa y Cervantes Forero, 2026) y se corren con un solo comando: `python tests/run_all.py`. Para cada suite se levanta de cero un clúster de pruebas aislado en Docker (base de datos vacía, ControlNode y tres DataNodes, con bloques de 1 MB y tiempos cortos), y las pruebas usan la API real, igual que el cliente. Cómo correrlas está en `tests/README.md`. La corrida completa, en un PC con Windows y Docker Desktop, tomó unos 5 minutos (Tabla 11; Figura 9).

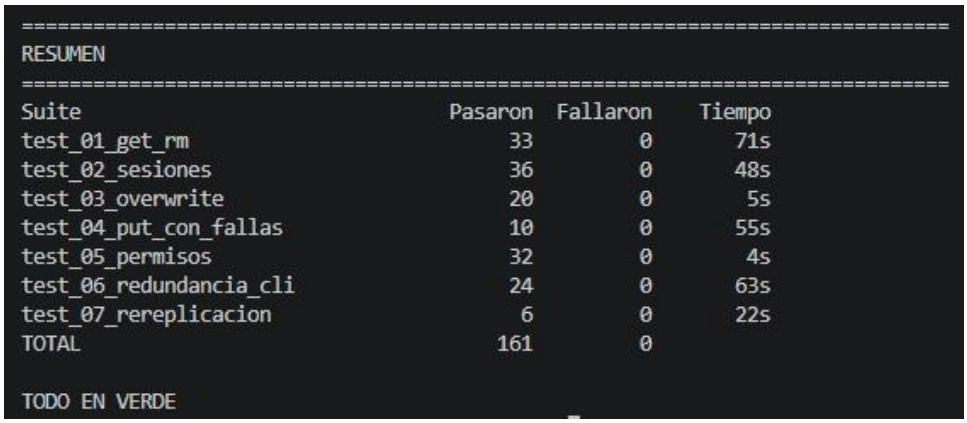
Tabla 11

Suites de pruebas automáticas y sus resultados

Suite (`tests/`)	Qué prueba	Resultado
`test_01_get_rm`	Descarga íntegra; GET con un DataNode apagado (usa la otra copia); copias dañadas detectadas por SHA-256; 503 sin copias; GET concurrentes; rm con un nodo caído y su limpieza posterior	33 / 33
`test_02_sesiones`	open, read, close; WRITE LOCK (423) y su vencimiento (410); 10 rondas de 3 `open` WRITE simultáneos; `write` que crea versión nueva; `write` que no logra factor 2 deja intacta la versión anterior	36 / 36
`test_03_overwrite`	409 sin overwrite; reemplazo de 3 a 5 bloques y de 5 a 1; borrado de la versión vieja; 423 con el archivo bloqueado; 403 sin permiso de escritura	20 / 20
`test_04_put_con_fallas`	Un DataNode cae sin que el monitor lo note: sus copias se guardan en otro nodo; sin factor 2 posible, el PUT y el overwrite fallan sin dejar rastro	10 / 10
`test_05_permisos`	chmod, chgrp, ACL de usuario y de grupo, grupos; solo el dueño cambia permisos; nadie se agrega solo a un grupo ajeno; las ACL se borran con su recurso	32 / 32
`test_06_redundancia_cli`	Bloques con una sola copia se restauran solos; copias huérfanas se borran y las recientes se respetan; comandos del cliente CLI	24 / 24
`test_07_rereplicacion`	Archivo de 40 bloques; al apagar un DataNode se recuperan todas las copias	6 / 6 (80 de 80 copias en 14,9 s)
Total		161 / 161

Figura 9

Resumen de la ejecución de las pruebas automáticas



```
================================================================================
RESUMEN
================================================================================
Suite                          Pasaron  Fallaron    Tiempo
test_01_get_rm                      33         0       71s
test_02_sesiones                    36         0       48s
test_03_overwrite                   20         0        5s
test_04_put_con_fallas              10         0       55s
test_05_permisos                    32         0        4s
test_06_redundancia_cli             24         0       63s
test_07_rereplicacion                6         0       22s
TOTAL                              161         0

TODO EN VERDE
```

Nota. Resumen de `python tests/run_all.py` en un PC con Windows y Docker Desktop: las 7 suites pasan, 161 de 161.

Durante la re-replicación de las 80 copias se consultó `/health` 78 veces y la respuesta más lenta fue de 0,07 s, porque la copia corre en segundo plano.

Errores que encontraron las pruebas

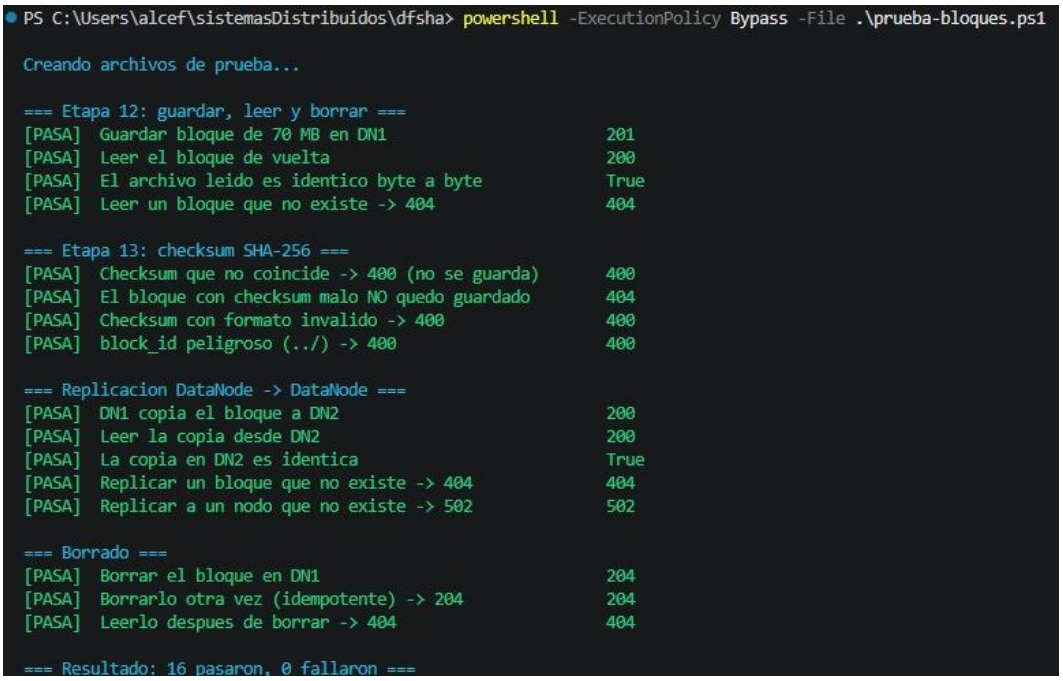
Al escribir y correr estas suites salieron tres errores reales, que corregimos antes de entregar:

- *Dos `open` WRITE al mismo tiempo podían chocar* (*deadlock* en PostgreSQL, respuesta 500). Cada uno creaba su handle antes de bloquear el archivo, y quedaban esperándose entre sí. Ahora se bloquea el archivo primero; la prueba de 10 rondas lo verifica.
- *El dueño podía quedar sin acceso a su propia carpeta* si le daba una ACL a un grupo del que él mismo es miembro, porque la ACL del grupo se revisaba antes que los bits del dueño. Ahora el dueño se revisa primero, como en las ACL POSIX.
- *Un overwrite que fallaba por DataNodes caídos podía responder 500 en vez de 422.* En Docker, conectarse a un DataNode apagado no falla al instante: tarda hasta el timeout de conexión. Mientras tanto se vencía el lock del overwrite, la limpieza lo borraba y el ControlNode fallaba al liberarlo. Ahora el lock se renueva antes de cada intento de enviar una réplica y se libera aunque ya no exista; la versión anterior del archivo siempre quedó intacta.

Además, durante el desarrollo corrimos scripts de prueba en PowerShell contra el clúster local en Docker (Figuras 10, 11 y 12):

Figura 10

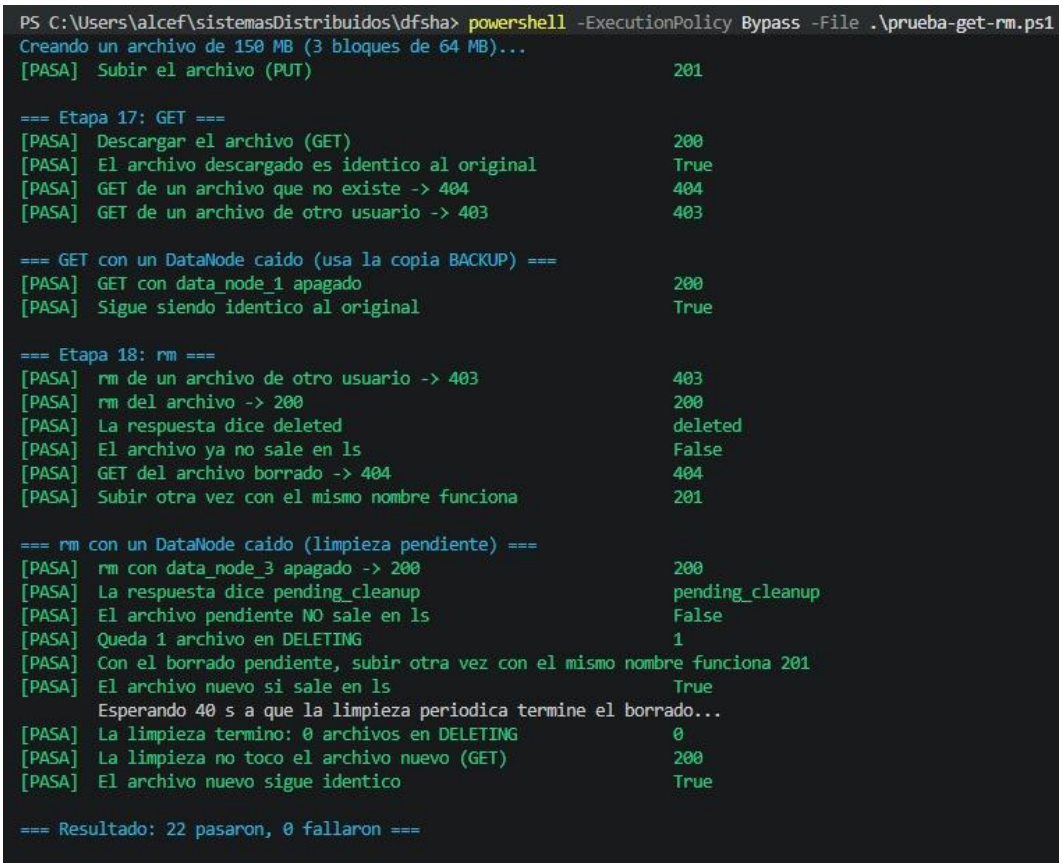
Prueba de la API de bloques del DataNode



Nota. API del DataNode: guardar, leer, checksum inválido rechazado, `block_id` peligroso rechazado, réplica entre DataNodes y borrado idempotente.

Figura 11

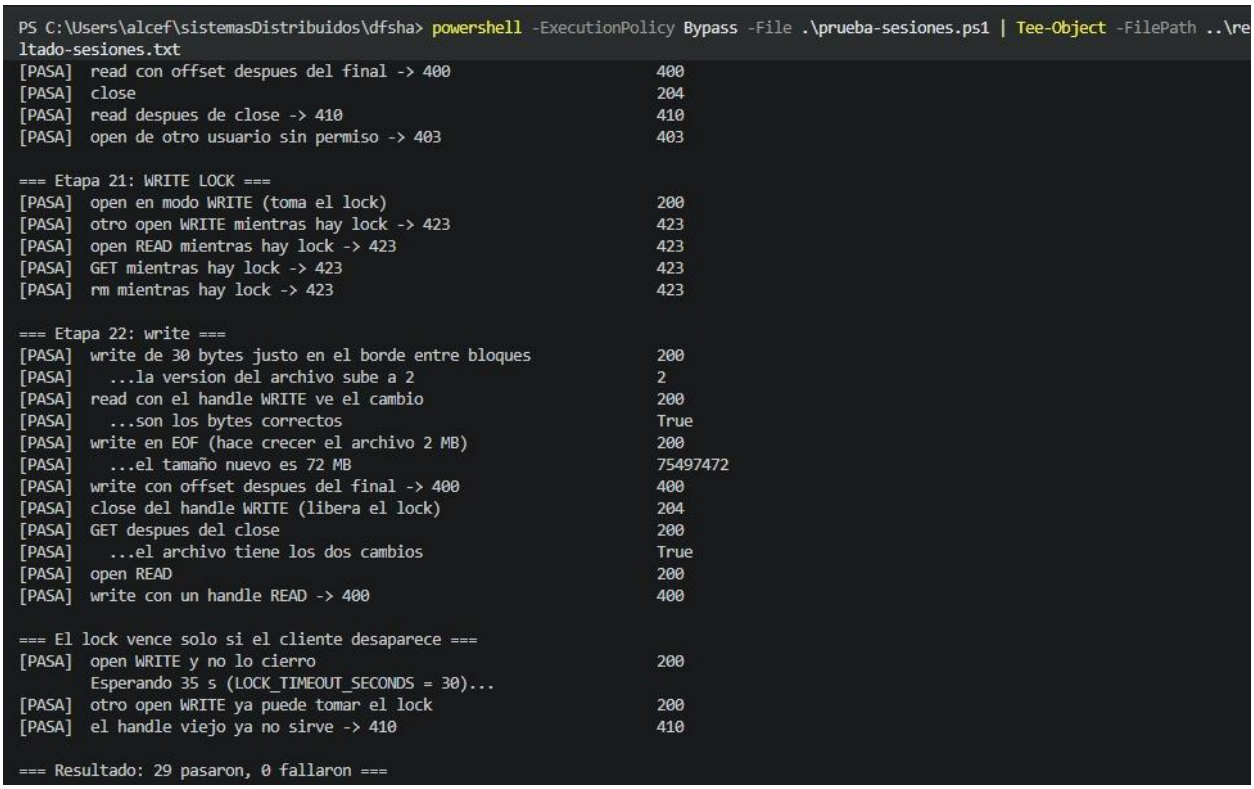
Prueba de las operaciones GET y rm



Nota. GET y rm, incluyendo GET con `data_node_1` apagado y rm con `data_node_3` apagado.

Figura 12

Prueba de las sesiones de archivo y del bloqueo de escritura



```
PS C:\Users\alcef\sistemasDistribuidos\dfsha> powershell -ExecutionPolicy Bypass -File .\prueba-sesiones.ps1 | Tee-Object -FilePath ..\re
ltado-sesiones.txt
[PASA]  read con offset despues del final -> 400                    400
[PASA]  close                                                       204
[PASA]  read despues de close -> 410                                410
[PASA]  open de otro usuario sin permiso -> 403                     403

=== Etapa 21: WRITE LOCK ===
[PASA]  open en modo WRITE (toma el lock)                           200
[PASA]  otro open WRITE mientras hay lock -> 423                    423
[PASA]  open READ mientras hay lock -> 423                          423
[PASA]  GET mientras hay lock -> 423                                423
[PASA]  rm mientras hay lock -> 423                                 423

=== Etapa 22: write ===
[PASA]  write de 30 bytes justo en el borde entre bloques           200
[PASA]    ...la version del archivo sube a 2                        2
[PASA]  read con el handle WRITE ve el cambio                       200
[PASA]    ...son los bytes correctos                                True
[PASA]  write en EOF (hace crecer el archivo 2 MB)                  200
[PASA]    ...el tamaño nuevo es 72 MB                               75497472
[PASA]  write con offset despues del final -> 400                   400
[PASA]  close del handle WRITE (libera el lock)                     204
[PASA]  GET despues del close                                       200
[PASA]    ...el archivo tiene los dos cambios                       True
[PASA]  open READ                                                   200
[PASA]  write con un handle READ -> 400                             400

=== El lock vence solo si el cliente desaparece ===
[PASA]  open WRITE y no lo cierro                                   200
        Esperando 35 s (LOCK_TIMEOUT_SECONDS = 30)...
[PASA]  otro open WRITE ya puede tomar el lock                      200
[PASA]  el handle viejo ya no sirve -> 410                          410

=== Resultado: 29 pasaron, 0 fallaron ===
```

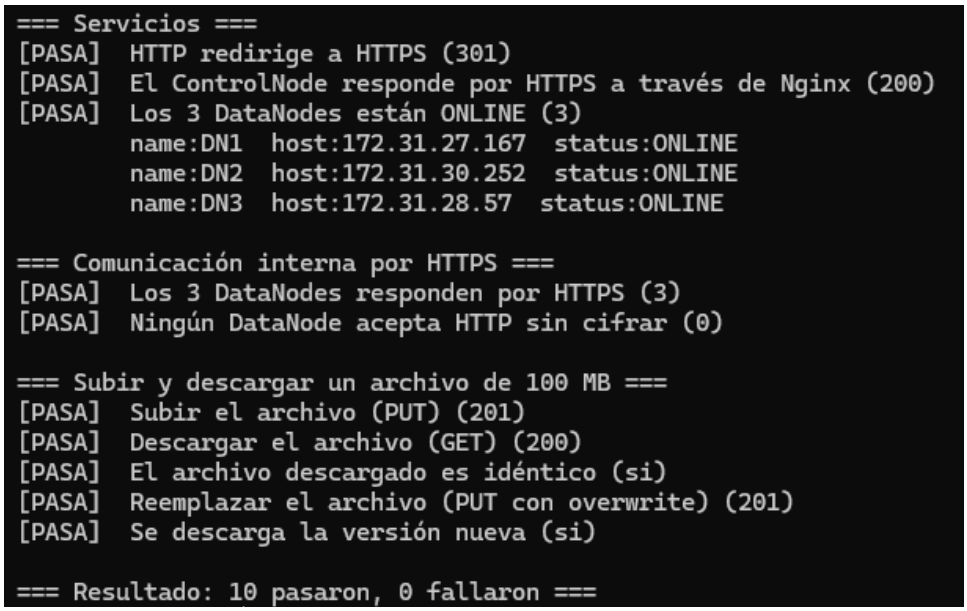
Nota. Sesiones de archivo y WRITE LOCK.

Prueba del despliegue en AWS

El script `deploy/prueba-despliegue.sh` corre en la VM1 y prueba el sistema de punta a punta, pasando por las 4 VMs. Resultado: 10 de 10 (Figura 13).

Figura 13

Resultado de la prueba del despliegue en AWS

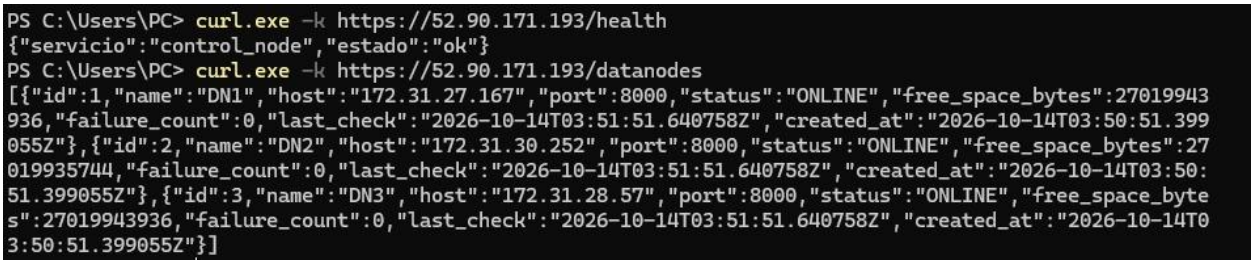


Nota. HTTP redirige a HTTPS, los 3 DataNodes ONLINE, los DataNodes rechazan HTTP sin cifrar, y un archivo de 100 MB sube, baja idéntico y se reemplaza con overwrite.

Desde un computador fuera de AWS, el sistema responde por internet con la IP pública de la VM1 (Figura 14):

Figura 14

Acceso externo al ControlNode por HTTPS



Nota. Acceso externo por HTTPS: el ControlNode responde y reporta los 3 DataNodes ONLINE con sus IPs privadas.

Caída de un DataNode en AWS

Procedimiento

Subir un archivo de 300 MB (5 bloques), contar los bloques en cada DataNode, apagar el contenedor de DN2, observar los estados y el log, volver a contar, descargar el archivo con DN2

todavía apagado y comparar su SHA-256 con el original. Hicimos la prueba dos veces y en las dos dio lo mismo.

Resultado

El sistema detectó la caída en 30 s, recuperó las 4 copias que estaban en DN2 copiándolas entre DN1 y DN3, y el archivo se descargó idéntico (Tabla 12).

Tabla 12

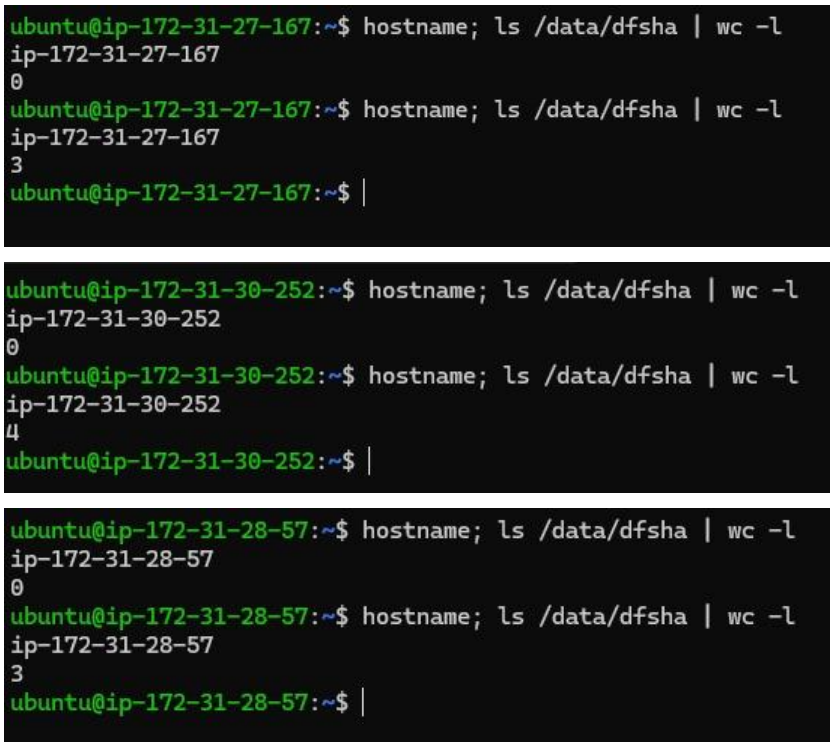
Copias por DataNode antes y durante la caída de DN2

DataNode	Copias antes	Copias con DN2 caído
DN1	3	5
DN2	4	apagado
DN3	3	5
Total	10	10

El reparto inicial (3, 4, 3) es exactamente el que predice Round Robin para 5 bloques (véase la sección Particionamiento, distribución y replicación). Después de la caída, las 10 copias quedan en los dos nodos vivos: ningún bloque se quedó con una sola copia (Figuras 15 y 16).

Figura 15

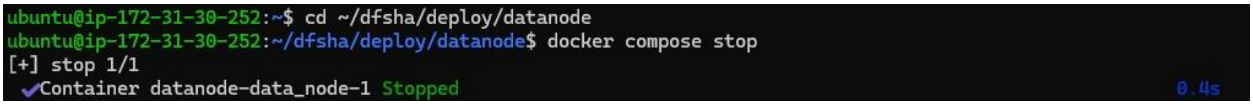
Bloques almacenados en cada DataNode antes de la caída



Nota. Bloques por nodo antes de la caída: DN1 (172.31.27.167) 3, DN2 (172.31.30.252) 4, DN3 (172.31.28.57) 3. El primer 0 es antes de subir el archivo.

Figura 16

Detención del contenedor de DN2



Nota. Se apaga el DataNode DN2.

Detección

Con chequeos cada 10 s y umbral de 3 fallos, DN2 pasó a SUSPECT en el primer chequeo fallido y a OFFLINE al tercero (Tabla 13; Figuras 17, 18 y 19):

Tabla 13

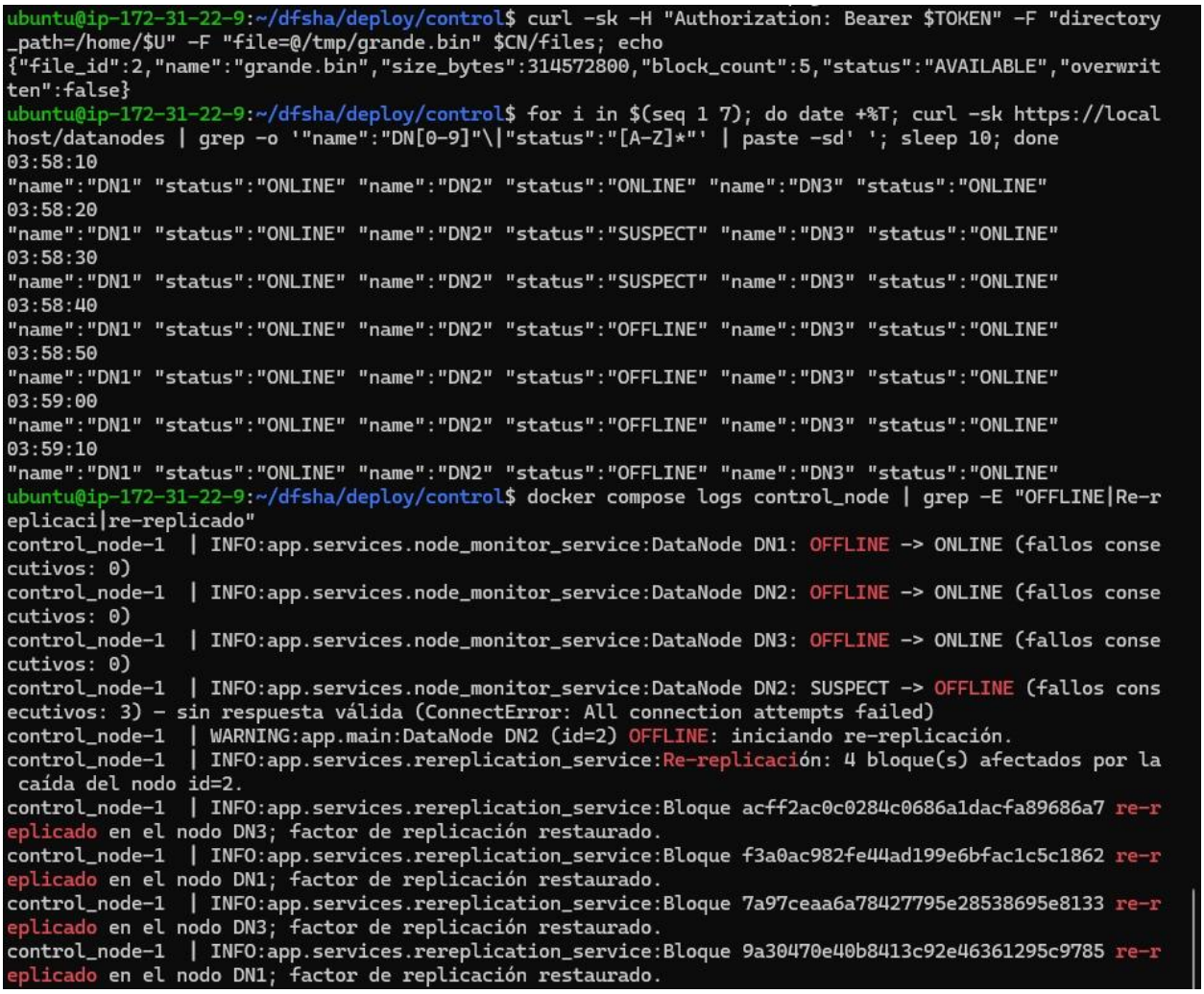
Estados de los DataNodes durante la caída de DN2

Hora (UTC)	DN1	DN2	DN3
03:58:10	ONLINE	ONLINE	ONLINE

Hora (UTC)	DN1	DN2	DN3
03:58:20	ONLINE	SUSPECT	ONLINE
03:58:30	ONLINE	SUSPECT	ONLINE
03:58:40	ONLINE	OFFLINE	ONLINE

Figura 17

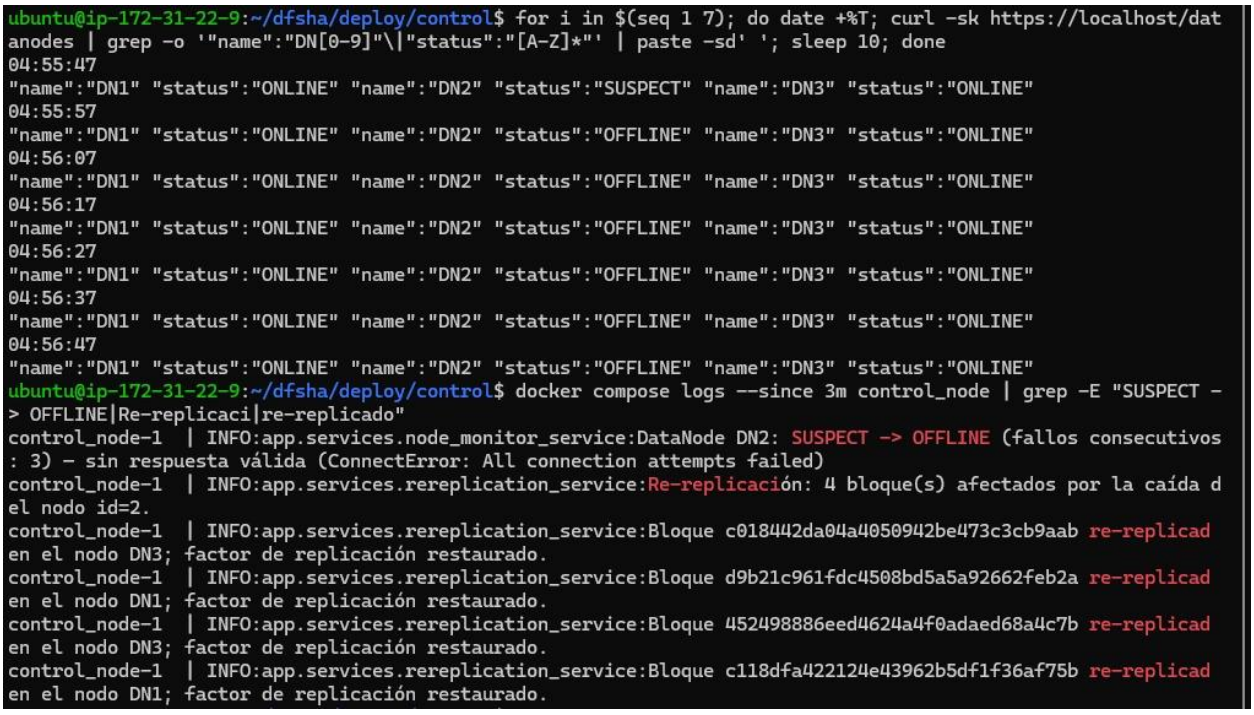
Estados de los DataNodes y registro de la re-replicación en la primera prueba



Nota. Primera prueba: DN2 pasa de ONLINE a SUSPECT y a OFFLINE; el log muestra "4 bloque(s) afectados" y cada bloque re-replicado en DN1 o DN3.

Figura 18

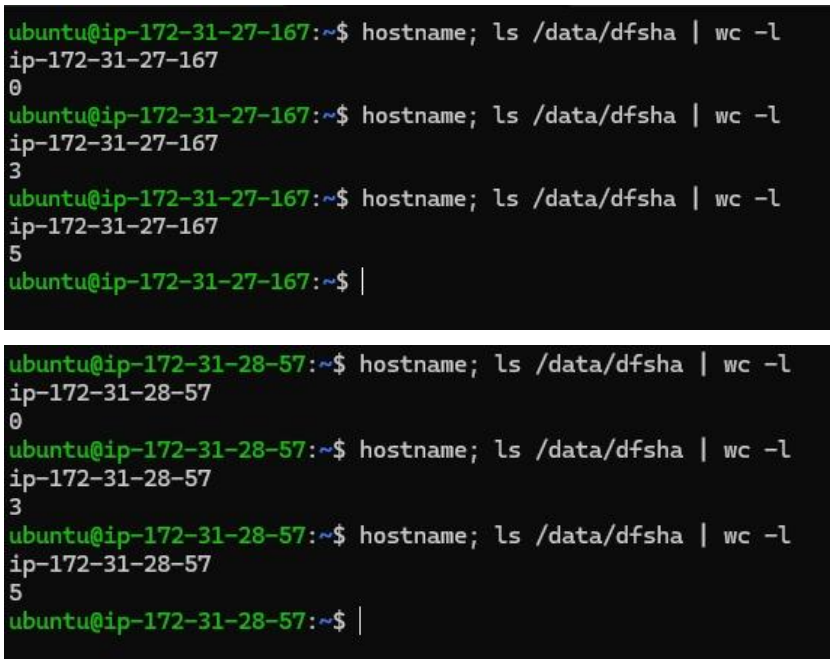
Estados de los DataNodes y registro de la re-replicación en la segunda prueba



Nota. Segunda prueba: el mismo resultado, otros 4 bloques recuperados (2 hacia DN1 y 2 hacia DN3).

Figura 19

Bloques almacenados en DN1 y DN3 después de la re-replicación



Nota. Después de la re-replicación: DN1 y DN3 tienen 5 copias cada uno.

Integridad

Con DN2 todavía apagado, el archivo descargado tiene el mismo SHA-256 que el original (`78d677873727f521ada6af2536efac69e96f093539f4b4f8389189cc01e69bcd`), y `cmp` no encuentra diferencias (Figura 20).

Figura 20

Comparación del SHA-256 del archivo original y del descargado



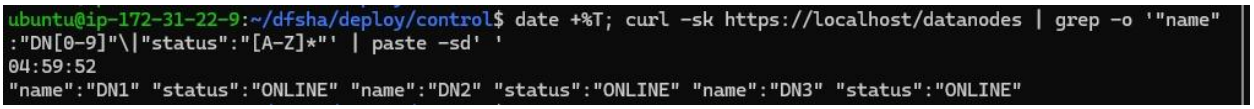
Nota. El archivo recuperado es idéntico byte a byte.

Recuperación

Al volver a encender DN2, el monitor lo marcó ONLINE en el siguiente chequeo. Sus copias viejas, que ya no están en los metadatos, las borra la limpieza de huérfanas pasados 15 minutos (Figura 21).

Figura 21

DataNodes en estado ONLINE después de la recuperación



Nota. Los tres DataNodes vuelven a estar ONLINE.

Análisis

- *Disponibilidad:* con un nodo de tres caído no se perdió ningún dato ni se interrumpió la descarga. El factor de replicación se recuperó solo, sin intervención manual.
- *Tiempo de detección:* 30 s, que sale de 3 chequeos × 10 s. Bajar el intervalo detecta antes pero aumenta las falsas alarmas por problemas de red pasajeros; por eso existe el estado SUSPECT.
- *Consistencia:* la descarga con un nodo caído, el overwrite y el `write` fallido siempre devolvieron una versión completa del archivo, nunca una mezcla.
- *Seguridad:* los DataNodes rechazaron HTTP sin cifrar, solo el puerto 443 está expuesto a internet, y los permisos respondieron 403 en todos los casos de acceso ajeno.
- *Distribución:* el reparto medido coincide con el calculado por Round Robin.

Limitaciones y trabajo futuro

Las principales limitaciones del sistema, la razón por la que se aceptaron y la mejora que se propone para cada una se resumen a continuación (Tabla 14).

Tabla 14

Limitaciones del sistema y trabajo futuro

Limitación	Por qué la aceptamos	Mejora futura
Un solo ControlNode con un solo PostgreSQL	Es el modelo de HDFS básico: la alta disponibilidad es de los datos. Si el ControlNode cae, no se pierde nada (bloques y metadatos están en disco), pero el servicio para hasta que vuelva	ControlNode en espera con PostgreSQL replicado (o Raft entre ControlNodes), como el NameNode en alta disponibilidad de HDFS
El tráfico de datos pasa por el ControlNode	Es quien cifra y descifra, y así los DataNodes no quedan públicos	Que el cliente lea y escriba directo en los DataNodes con tokens de acceso temporales, cifrando del lado del cliente
`write` reemplaza el archivo completo	Da consistencia sin locks por bloque	Escribir solo los bloques que cambian, con versión por bloque

Limitación	Por qué la aceptamos	Mejora futura
Lock por archivo, no por bloque	Más simple y sin interbloqueos	Locks por rango de bytes
Certificado autofirmado hacia el cliente	Uso académico, sin dominio propio	Certificado de una autoridad (Let's Encrypt) con un dominio
Sin segundo factor (2FA)	Usuario, contraseña y JWT cubren lo pedido	TOTP como segundo factor y API keys por usuario
Reparto Round Robin sin mirar carga	Reparto parejo y predecible	Estrategia por espacio libre o carga (la interfaz ya lo permite)

Conclusiones

- *Se cumplió el objetivo.* DFSha guarda archivos grandes repartidos en bloques entre varios nodos, con 2 copias de cada bloque, y funciona desplegado en 4 máquinas de AWS Academy: la prueba de despliegue pasó 10 de 10 y las 7 suites automáticas pasaron 161 de 161.
- *La alta disponibilidad de los datos es real, no teórica.* Al apagar un DataNode en AWS, el sistema lo detectó en 30 s, recuperó solo las 4 copias que se perdieron y el archivo de 300 MB se descargó idéntico (mismo SHA-256), sin intervención manual.
- *Versiones nuevas en vez de escribir bloques en el sitio.* Reemplazar el archivo por una versión nueva que solo se activa cuando está completa hizo fácil garantizar consistencia, a cambio de reescribir el archivo completo en cada `write`.
- *La seguridad se pensó por capas.* Cifrado en el disco y en todos los saltos de red, autenticación con JWT, permisos Unix con ACL y grupos, y DataNodes que no quedan expuestos a internet. Derivar el certificado interno de la clave compartida evitó copiar certificados entre máquinas.
- *Las pruebas automáticas valieron la pena.* Encontraron tres errores que a mano no habíamos visto (un *deadlock* con dos `open` simultáneos, un caso en que el dueño perdía acceso a lo suyo y un overwrite que respondía 500 cuando un DataNode tardaba en fallar) y los dejamos corregidos.
- *Lo que sigue.* El punto débil es el ControlNode único: replicarlo (por ejemplo, con un PostgreSQL replicado y un ControlNode en espera) sería el siguiente paso para que todo el servicio, y no solo los datos, sea de alta disponibilidad.

Referencias

Garnica Roa, A. F. y Cervantes Forero, A. D. (2026). *DFSha: sistema de archivos distribuido con alta disponibilidad* [Software]. GitHub. https://github.com/afgarnicar/dfsha

Ghemawat, S., Gobioff, H. y Leung, S.-T. (2003). The Google file system. En *Proceedings of the Nineteenth ACM Symposium on Operating Systems Principles* (pp. 29–43). Association for Computing Machinery. https://doi.org/10.1145/945445.945450

Shvachko, K., Kuang, H., Radia, S. y Chansler, R. (2010). The Hadoop distributed file system. En *2010 IEEE 26th Symposium on Mass Storage Systems and Technologies (MSST)* (pp. 1–10). IEEE. https://doi.org/10.1109/MSST.2010.5496972

Universidad EAFIT. (2026). *Proyecto 1: diseño e implementación de un sistema de archivos distribuido con alta disponibilidad, rendimiento y seguridad (DFSha)* [Enunciado de proyecto]. ST0263 Tópicos Especiales en Telemática / SI3007 Sistemas Distribuidos.